Reviewer Pack — Eigenvalue Gap in SOS Moment Matrices for Max-CUT Approximat…

Entry b99dacf3b7fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 03:17:45 UTC

Eigenvalue Gap in SOS Moment Matrices for Max-CUT Approximation

Entry ID: `b99dacf3b7fe` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 03:17:45 UTC

1. Conjecture

Field A (mathematical branch): REAL_ALGEBRAIC_GEOMETRY **Field B** (complexity object): SUM_OF_SQUARES_HIERARCHY

Statement:

> For any max-CUT instance on n vertices, if there exists a 0.879-approximator, then the corresponding degree-d SOS moment matrix M must satisfy that all eigenvalues of M lie within [−1, 1]. If M has an eigenvalue outside this interval, then no such approximator exists. This property P(M) is preserved under polynomial reductions.

Rationale (proposer's reasoning):

> The SOS hierarchy's moment matrices encode correlations between variables in the relaxation. A 0.879-approximator for max-CUT requires the matrix to capture the graph's structure tightly, restricting eigenvalues to [−1, 1]. Violating this bound would imply the relaxation cannot achieve the desired approximation, aligning with Bouland-Kothari's SOS lower bounds for planted clique. The property is reduction-invariant because reductions preserve the problem's algebraic structure.

Taxonomy category: `SOS_DEGREE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `da4bc483f56da939`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        graph = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    graph[i][j] = graph[j][i] = random.randint(1, 10)
        return graph

    def matrix_multiplication(A, B):
        n = len(A)
        result = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    result[i][j] += A[i][k] * B[k][j]
        return result

    def gaussian_elimination(A, b):
        n = len(A)
        M = [A[i] + [b[i]] for i in range(n)]
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(M[j][i]) > abs(M[max_row][i]):
                    max_row = j
            M[i], M[max_row] = M[max_row], M[i]
            factor = M[i][i]
            for j in range(n):
                M[i][j] /= factor
            b[i] /= factor
            for j in range(n):
```

```
                if i != j:
                    factor = M[j][i]
                    for k in range(n):
                        M[j][k] -= factor * M[i][k]
                    b[j] -= factor * b[i]
        return [row[:-1] for row in M], b

    def compute_eigenvalues(matrix):
        n = len(matrix)
        identity = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
        eigenvalues = []
        for _ in range(10):  # Power iteration method
            x = [random.random() for _ in range(n)]
            x = [x[i] / sum(x) for i in range(n)]
            y = matrix_multiplication(matrix, x)
            lambda_ = sum(y[i] * x[i] for i in range(n))
            eigenvalues.append(lambda_)
        return eigenvalues

    n = random.randint(5, 40)
    graph = generate_random_graph(n)

    # Convert graph to moment matrix
    M = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(i + 1, n):
            M[i][j] = M[j][i] = graph[i][j]

    # Add identity matrix to make it a moment matrix
    for i in range(n):
        M[i][n] = M[n][i] = 1

    eigenvalues = compute_eigenvalues(M)

    metric_value = max(eigenvalues) - min(eigenvalues)
    conjecture_holds = all(-1 <= e <= 1 for e in eigenvalues)
    counterexample = "mapping_undefined" if not conjecture_holds else ""

    return {
        "metric_name": "Eigenvalue Gap",
        "metric_value": metric_value,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27a96c15.py", line 104, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27a96c15.py", line 84, in run_trial
    eigenvalues = compute_eigenvalues(M)
                  ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27a96c15.py", line 66, in compute_eigen
    y = matrix_multiplication(matrix, x)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27a96c15.py", line 35, in matrix_multip
    result[i][j] += A[i][k] * B[k][j]
                              ~~~~^^^
TypeError: 'float' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to a type error in matrix operations, preventing data collection. The pre-registered support condition cannot be evaluated without successful runs. | next: Debug the matrix multiplication code to handle float values properly and rerun tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	82087
2	preregistration	ollama_remote	qwen3:8b	0	0	28124
3	novelty	ollama_remote	qwen3:8b	0	0	24873
4	novelty	ollama_remote	qwen3:8b	0	0	19188
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15118
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11803
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13301
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12447
9	judge	ollama_remote	qwen3:8b	0	0	19768

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 226711 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b99dacf3b7fe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b99dacf3b7fe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b99dacf3b7fe.tar.gz` (if generated)